

ABM BUSINESS LOGIC BIBLE

SECTION 3 — MODULES & WORKFLOWS

Full 5-Whys Excavation · 14 Modules · The Verbs that Operate on the 37 Entities

Section 2 defined the nouns (37 entities, 6 tiers). Section 3 defines the verbs — the modules and workflows that create, read, and transform them. Each module is excavated via full 5-Whys and documented in engineering-handoff priority order: (1) step-by-step workflow logic, (2) entities consumed and written, (3) decision nodes and gates, (4) module boundary and orchestration.

Modules	14	5-Whys	70
New rules	70	Pipeline gates	M0·M4·M8·M10

MODULE ARCHITECTURE & PIPELINE ORDER

The original 9 modules expand to 14 once mapped against the 6-tier entity model. The mandatory pipeline (enforced by orchestration, ORCH-PIPELINE-001) runs: detect → bless → score → enrich → reason → generate → review (if gated) → send → engage → learn. Three new modules emerged from the entity model — Timing (M5), Portfolio (M6), and Strategic Memory (M12) — because Buying_Window, Portfolio_Allocation, and the Tier 6 entities each need a clear owner.

M0	Intelligence Governance	Gates data quality / confidence (blesses or quarantines)
M1	Signal Intelligence	Detects + structures external signals
M2	Scoring & Prioritization	Three scores + master equations
M3	Enrichment & Intelligence	Deep profiles + buying committee
M4	Account Strategy & Reasoning	Reasoning chain + standing strategy
M5	Timing & Window	Buying_Window lifecycle (now vs later)
M6	Portfolio Allocation	Capital across accounts (EROA + dependency)
M7	AI Generation & QC	Copy generation + hallucination/tone gate
M8	Outreach Execution	Paced multi-channel sending + exit logic
M9	Engagement Tracking	Recipient actions as inbound signals
M10	Human Handoff & Review	Handoff + AI-autonomy calibration
M11	Learning & Analytics	Attribution + bounded recalibration
M12	Strategic Memory	Tier 6 judgment (DNA, narrative, regret, dependency)
M-Orch	Orchestration (n8n)	Routing/scheduling/integration — no business logic

M0 — INTELLIGENCE GOVERNANCE ENGINE

The meta-module that gates the quality, freshness, and confidence of all intelligence before any other module is allowed to act on it. Nothing downstream trusts data M0 has not blessed.

WHY #1 — Why does a governance module need to exist BEFORE signal/scoring/enrichment?

Because every downstream decision inherits the quality of its inputs. If a stale, duplicated, or low-confidence signal flows straight into scoring, the error propagates through strategy, message, and brand spend. M0 is the gatekeeper that computes the Intelligence Confidence Score (the third score) and quarantines anything that fails quality thresholds before it can contaminate the pipeline.

NEW RULE — M0-GATE-001: No intelligence (signal, enrichment, engagement) may be consumed by a downstream module until M0 has assigned it an Intelligence Confidence Score and passed it through freshness, dedup, and provenance checks. Unblessed data is quarantined, never silently used.

WHY #2 — Why compute confidence as a SCORE rather than a pass/fail flag?

Because data quality is a gradient, not a binary. A two-week-old firmographic fact is usable but weaker than a fresh one. A score lets downstream modules weight intelligence proportionally (ICS gates intent in the opportunity equation) instead of crudely accepting or rejecting it.

NEW RULE — M0-CONF-001: M0 computes a three-dimensional Intelligence Confidence Score (source reliability x freshness x corroboration) per intelligence item. The score flows into Three_Score_Record as the confidence dimension and proportionally weights the item everywhere it is used.

WHY #3 — Why route conflicts and gaps through M0 rather than each module handling its own?

Because consistency requires one authority. If every module resolves conflicting enrichment its own way, the system contradicts itself. M0 centralizes conflict resolution and gap declaration so the whole system reasons from one reconciled truth and one honest list of what it does not know.

NEW RULE — M0-RECONCILE-001: M0 owns intelligence-conflict reconciliation (via ENRICH-RECONCILE-001) and Capability_Gap declaration. When sources disagree or data is missing, M0 records the conflict/gap and emits a confidence-lowered or quarantined result rather than letting modules guess independently.

WHY #4 — Why must M0 be event-driven rather than a periodic batch cleaner?

Because timing is the primary weapon — a hot signal blessed six hours late is a missed window. M0 must evaluate intelligence the moment it arrives so downstream reaction is immediate, while still catching the staleness of older items at read time.

NEW RULE — M0-EVENT-001: M0 evaluates each intelligence item on arrival (event-driven), emitting a System_Event on bless/quarantine. It also re-evaluates confidence at read time so decay since ingestion is always reflected. No fixed-interval batch gate is permitted for hot signals.

WHY #5 — Deepest reason M0 exists?

Because the system's single greatest risk is acting confidently on bad information — the hallucination/staleness/metric-manipulation failure family. M0 is the institutional discipline that the system never mistakes noise for knowledge. It is the foundation of trust for everything above it.

NEW RULE — M0-PRIMACY-001: M0 is the first module in the pipeline and an immutable governance authority. Its confidence scores and quarantine decisions cannot be overridden by downstream modules; only a governed human action can release quarantined intelligence.

① Workflow Logic (step-by-step)

1. Intelligence item arrives (signal / enrichment / engagement) → emit ingest System_Event
2. Provenance check: is raw_payload + source attribution present? (SIGNAL-PROVENANCE / ENRICH-RECORD)
3. Dedup check: compute event_fingerprint; collapse to corroborating_sources if matched
4. Freshness check: apply decay/staleness window for the item type

5. Confidence computation: source reliability x freshness x corroboration → Intelligence Confidence Score
6. Conflict check: if sources disagree, run reconciliation; retain both + rationale
7. Decision: PASS (bless, write confidence, emit blessed event) / QUARANTINE (hold, declare gap if needed)
8. Blessed item released to downstream module subscribers

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
- Signal (raw_payload, fingerprint) - Enrichment (sources) - Engagement_Event - Rule_Registry (thresholds, weight versions) - Market_Interaction_History (for contact-level confidence)	- Three_Score_Record.intelligence_confidence - Enrichment.conflict_resolution - Capability_Gap - System_Event (bless/quarantine) - Unresolved_Intelligence (quarantine holding)

③ Decision Nodes & Gates

Decision node	Branch logic
Provenance present?	NO → quarantine + declare DATA gap; YES → continue
Duplicate fingerprint?	YES → collapse into existing signal, raise corroboration confidence (not weight); NO → continue
Confidence >= threshold?	NO → quarantine or release as low-confidence advisory; YES → bless
Sources conflict on high-stakes field?	YES → reconcile; if unresolved, lower confidence + flag human review

④ Module Boundary & Orchestration

M0 owns data-quality governance and the confidence dimension ONLY. It does NOT score opportunity (M2), interpret meaning (M4), or decide action. It blesses or quarantines; it never sells. Orchestration: M0 sits at pipeline head; n8n routes all raw intelligence through M0 before any subscriber fires.

M1 — SIGNAL INTELLIGENCE

Continuously monitors external sources, detects buying-intent signals, normalizes and stores them as the event atoms that drive the entire system.

WHY #1 — Why does signal detection need to be a continuous module rather than an on-demand lookup?

Because the system is event-driven and timing is the weapon. Opportunities open and close on the market's schedule, not the system's. Continuous polling/ingestion means the system reacts when the world changes, not when someone remembers to look. A missed signal is a missed window.

NEW RULE — M1-CONTINUOUS-001: M1 polls configured sources on type-specific schedules (high-priority sources more frequently) and ingests push sources in real time. Polling cadence per source is configurable in Rule_Registry; hot signal types get the shortest intervals.

WHY #2 — Why normalize signals into one schema rather than store source-native formats?

Because downstream modules must reason uniformly. A LinkedIn hire, a SAMA regulatory notice, and a funding round are different shapes but must all become comparable Signal entities with type, priority, and fingerprint. Normalization is what makes cross-source intelligence possible.

NEW RULE — M1-NORMALIZE-001: M1 normalizes every raw source payload into the Signal schema (signal_type, priority, raw_payload, parsed_interpretation, fingerprint) with parser_version stamped. Raw payload is always retained for reparsing under future parser versions.

WHY #3 — Why resolve entities (which account is this about?) inside M1 rather than later?

Because a signal with no account is noise. Entity resolution (matching 'SNB', 'Saudi National Bank', Arabic name, aliases) at ingestion attaches the signal to the right Account or, failing that, to Unresolved_Intelligence — never silently dropped or misattributed.

NEW RULE — M1-RESOLVE-001: M1 performs entity resolution against Account.aliases. Resolved signals attach to the Account; unresolved signals go to Unresolved_Intelligence for later resolution (per ACCT-IDENTITY-005). Misattribution is worse than non-attribution — ambiguous matches stay unresolved pending confidence.

WHY #4 — Why does M1 hand off to M0 rather than directly trigger scoring?

Because raw detection is not blessed intelligence. M1 finds and structures; M0 judges quality. Separating detection from governance prevents a noisy or duplicate signal from triggering a scoring cascade before its confidence is established.

NEW RULE — M1-HANDOFF-001: M1 emits each normalized signal to M0 for governance, not directly to M2 scoring. Scoring is triggered only after M0 blesses the signal. M1 never assigns confidence or triggers downstream action itself.

WHY #5 — Deepest reason M1 exists?

Because the system can only be as aware as its senses. M1 is the sensory organ — without continuous, structured, attributed perception of the market, every other module is reasoning in the dark. It is the physical embodiment of event-driven architecture.

NEW RULE — M1-PRIMACY-001: Every account entry and advancement traces to a Signal produced by M1 (per SIGNAL-PRIMACY-001). M1 is the sole authorized producer of externally-sourced Signal entities; other modules may produce engagement-derived signals but not external ones.

① Workflow Logic (step-by-step)

1. Cron/push trigger fires for a configured source (cadence from Rule_Registry)
2. Fetch external data (News, LinkedIn, SAMA RSS, funding, hiring, job boards)
3. Parse & normalize into Signal schema; stamp parser_version
4. Entity resolution against Account.aliases → Account or Unresolved_Intelligence
5. Compute event_fingerprint for dedup
6. Persist Signal (immutable raw_payload + parsed_interpretation)
7. Emit Signal to M0 for governance (NOT directly to scoring)

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
• External sources (APIs/RSS/scrape) • Account.aliases (entity resolution) • Rule_Registry (poll cadence, signal_type → priority, decay curves)	• Signal • Unresolved_Intelligence • System_Event (signal detected)

③ Decision Nodes & Gates

Decision node	Branch logic
Source reachable / quota ok?	NO → log, backoff, retry per source policy; YES → fetch
Parseable into schema?	NO → store raw + flag parse failure (not dropped); YES → normalize
Account resolved with confidence?	NO → Unresolved_Intelligence; YES → attach to Account
Within freshness/expiry?	Expired-on-arrival → store but mark zero-weight (no trigger)

④ Module Boundary & Orchestration

M1 detects and structures external signals ONLY. It does NOT judge quality (M0), score (M2), or interpret strategic meaning (M4). Engagement signals come from M9, not M1. Orchestration: n8n owns the polling schedule and routing; M1's detection logic lives in a service, not in n8n (per n8n philosophy — no business logic in orchestration).

M2 — SCORING & PRIORITIZATION

Computes the three independent scores and the master equations, turning blessed intelligence into a ranked, explainable priority for every account.

WHY #1 — Why is scoring a dedicated module rather than a function called ad hoc?

Because scoring must be consistent, versioned, explainable, and retriggerable on every relevant event. A dedicated module enforces one scoring path, one weight-set version per run, and one stored snapshot — preventing the black-box, untraceable scoring that the Section 1 pitfalls warn against.

NEW RULE — M2-CENTRAL-001: M2 is the sole authority that writes `Three_Score_Record`. All prioritization across the system reads M2's stored scores; no module computes its own private score. Every run stamps the `weight_set_version` used.

WHY #2 — Why compute three scores and two equations rather than a single priority number?

Because fit, intent, and confidence are orthogonal, and opportunity vs. decision are different questions. The Effective Opportunity equation answers 'how attractive is this?'; the Decision Score answers 'should we spend capital now?'. Collapsing them hides why an account ranks where it does and lets cheap-but-weak beat expensive-but-strong incorrectly.

NEW RULE — M2-EQUATION-001: M2 computes $\text{Effective Opportunity} = \text{Dynamic} \times (\text{ICS}/100) \times \text{Stage} \times \text{Budget} \times \text{Entrenchment} \times \text{Risk} \times \text{Window}$, then $\text{Decision Score} = \text{Effective Opportunity} \times (\text{Readiness}/100) / \text{Capital Cost}$. Both are stored with full dimension breakdown. ICS gates intent so low-confidence cannot inflate priority.

WHY #3 — Why must M2 retrigger on events rather than run on a schedule?

Because a stale ranking misallocates capital. When a new signal is blessed, a window opens, engagement arrives, or readiness changes, the affected account's score must immediately recompute so the priority queue always reflects current reality.

NEW RULE — M2-RETRIGGER-001: M2 subscribes to events (blessed signal, window state change, engagement, readiness/strategy change, org-assessment change) and recomputes the affected account's scores on each. `Account.current_state` cache is updated only by M2, emitting a `System_Event` per recompute.

WHY #4 — Why store the full dimension breakdown on every run?

Because explainability is non-negotiable — a human reviewer and an audit must be able to see why an account is HOT. The breakdown also enables monthly recalibration: you cannot improve weights you cannot trace.

NEW RULE — M2-EXPLAIN-001: Every M2 run writes per-dimension contributions (input, raw value, weight, contribution) into `Three_Score_Record`. A score without breakdown is invalid and rejected. Breakdowns feed M11 recalibration.

WHY #5 — Deepest reason M2 exists?

Because attention is the binding constraint and must be invested where return is highest. M2 is the system's prioritization conscience — the module that decides, transparently and consistently, what deserves the system's finite capital next. It converts knowledge into ranked intent.

NEW RULE — M2-PRIMACY-001: No sequencing, allocation, or outreach may reference a priority not produced and stored by M2. Rankings on ephemeral or module-local scores are prohibited (per SCORE-PRIMACY-001).

① Workflow Logic (step-by-step)

1. Subscribed event received (blessed signal / window change / engagement / readiness change)

2. Load account intelligence: signals (weighted by decay+confidence), enrichment, window, stage, budget, entrenchment, org-assessment
3. Compute Fit (ICS), Intent (Dynamic), Intelligence Confidence dimensions
4. Compute Effective Opportunity (apply all modifiers) with dimension breakdown
5. Compute Decision Score (apply readiness gate ÷ capital cost)
6. Write immutable Three_Score_Record with weight_set_version + breakdown
7. Update Account.current_state cache (tier/stage/readiness); emit System_Event
8. If tier crosses HOT threshold → emit event for M3 enrichment + M4 strategy

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
• Signal (blessed, weighted) • Enrichment • Buying_Window (multiplier) • Vendor_Intelligence (entrenchment) • Organizational_Assessment (readiness) • Initiative (stage/criticality) • Risk_Assessment (risk modifier) • Organizational_DNA (conditioning) • Rule_Registry (weight_set_version)	• Three_Score_Record • Account.current_state • System_Event (rescored, tier change)

③ Decision Nodes & Gates

Decision node	Branch logic
Intelligence Confidence below floor?	YES → score produced but flagged low-confidence; gates aggressive action downstream
Readiness < 40?	YES → Decision Score suppressed (DO NOT ENGAGE band per PATIENCE-001)
Negative/zero EROA inputs?	Flag for M6 exclusion this cycle
Tier crosses HOT?	YES → trigger enrichment (M3) + strategy (M4)

④ Module Boundary & Orchestration

M2 scores and ranks ONLY. It does NOT gather intelligence (M1/M3), decide capital allocation across the portfolio (M6), or choose actions (M4). It consumes modifiers (window, risk, entrenchment) computed by other modules; it does not compute them itself. Orchestration: event-subscriber service; n8n routes trigger events to M2's scoring API.

M3 — ENRICHMENT & INTELLIGENCE

Builds deep intelligence profiles for high-priority accounts and their buying committees — converting a bare account into an actionable, sourced intelligence object.

WHY #1 — Why trigger enrichment on score threshold rather than enriching everything?

Because enrichment costs capital (API spend, rate limits) and most accounts never warrant it. Enriching on HOT-tier detection focuses scarce enrichment budget where opportunity is real, per the EROA discipline.

NEW RULE — M3-TRIGGER-001: M3 enriches on M2 HOT-tier detection (or explicit human request). Speculative enrichment of COLD accounts is rate-limited and requires justification. Enrichment scope scales with Decision Score — higher score earns deeper enrichment.

WHY #2 — Why does M3 build the buying committee, not just company firmographics?

Because organizations don't buy, committees do (Account-Centric Design). Firmographics alone can't tell you who the economic buyer, technical evaluator, or anti-champion is. M3 must surface the people and their roles to make account strategy possible.

NEW RULE — M3-COMMITTEE-001: M3 identifies and builds Buying_Committee_Member records (economic buyer, technical evaluator, champion, blocker) with verified contacts. An account enriched without at least an attempt at committee mapping is flagged incomplete (per BCM-PRIMACY-001).

WHY #3 — Why route all enriched facts through M0 governance?

Because enrichment is the largest source of stale and conflicting data. A fact from Apollo and a contradicting one from Clearbit must be reconciled with provenance, not silently overwritten. M0 blessing keeps the intelligence base honest.

NEW RULE — M3-GOVERN-001: M3 submits enriched facts to M0 for provenance, freshness, and conflict reconciliation before they are usable. Each fact carries source + source_date; conflicts are reconciled by source reliability with both values retained (ENRICH-RECONCILE-001).

WHY #4 — Why does M3 feed both scoring (M2) and strategy (M4)?

Because enrichment changes both how attractive an account is and how to approach it. New committee data and initiative intelligence retrigger scoring AND inform the reasoning chain. M3 is the bridge that deepens both prioritization and strategy.

NEW RULE — M3-FEED-001: On enrichment completion, M3 emits events that retrigger M2 scoring (new dimensions available) and notify M4 (committee + initiative + vendor intelligence ready for strategy). Enrichment is not complete until these events fire.

WHY #5 — Deepest reason M3 exists?

Because the system must convert the unknown into the known before it can act wisely. M3 is the research faculty — the difference between reaching out to a name and reaching out to a understood organization with a mapped committee and a known initiative. It earns the right to personalize.

NEW RULE — M3-PRIMACY-001: No persona inference, pain hypothesis, or committee strategy may be built on facts lacking M3-sourced, M0-blessed provenance (per ENRICH-PRIMACY-001). Unsourced claims are hypotheses, not evidence.

① Workflow Logic (step-by-step)

1. HOT-tier event from M2 (or human request) received
2. Determine enrichment depth from Decision Score (capital-scaled)
3. Fetch company/firmographic data (Apollo, Clearbit, Clay, LinkedIn)
4. Identify decision-makers → build Buying_Committee_Member records
5. Verify contact emails; resolve reachability (email vs LinkedIn-only)
6. Gather initiative, vendor/incumbent, and org-stability intelligence
7. Submit all facts to M0 (provenance, freshness, conflict reconciliation)
8. On bless: persist Enrichment + linked entities; emit retrigger (M2) + ready (M4) events

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
------------------	-------------------

• Account (target) • Three_Score_Record (depth scaling) • External enrichment sources • Market_Interaction_History (prior contact) • Rule_Registry (cost budgets, depth tiers)	• Enrichment • Contact • Buying_Committee_Member • Initiative • Vendor_Intelligence • Organizational_Assessment • Relationship_Edge • System_Event
--	---

③ Decision Nodes & Gates

Decision node	Branch logic
Account at/above enrichment threshold?	NO → skip (or require justification); YES → enrich
Capital budget available for depth?	NO → reduce depth or queue; YES → proceed at scaled depth
Contact reachable (verified email)?	NO → LinkedIn-only eligibility flag; YES → email eligible
Committee mapped?	NO → flag incomplete, lower strategy confidence; YES → ready for M4

④ Module Boundary & Orchestration

M3 gathers and structures account/people intelligence ONLY. It does NOT bless its own data (M0), score (M2), or devise strategy (M4) — it supplies the raw material for them. Orchestration: triggered by M2 events via n8n; enrichment-source API calls and committee logic live in services.

M4 — ACCOUNT STRATEGY & REASONING

Owns the structured reasoning chain and the standing account strategy — the module where blessed intelligence becomes a coherent, defensible plan before any message is written.

WHY #1 — Why is strategy/reasoning a distinct module rather than part of message generation?

Because Principle 5 forbids generating messages from raw data — reasoning must precede and be separable from generation. A distinct module enforces the mandatory reasoning chain and maintains the account-level strategy that individual messages serve, preventing incoherent, tactic-by-tactic outreach.

NEW RULE — M4-SEPARATE-001: M4 produces the Reasoning_Chain and Account_Strategy BEFORE M7 generates any message. Generation without a complete, M4-produced reasoning chain is rejected (per REASON-PRIMACY-001 and MESSAGE-TRACE-001).

WHY #2 — Why must M4 maintain a standing Account_Strategy above individual reasoning chains?

Because sequences are tactics and the account needs one coherent strategy they all serve. Without a standing strategy, two touches can contradict each other and the account experiences an incoherent Decimal. The strategy holds the angle, posture, committee targets, connector path, and readiness gates.

NEW RULE — M4-STRATEGY-001: M4 maintains one active Account_Strategy per engaged account (value angle, competitive posture, target committee members, connector path, readiness gates, anchor initiative/window). All Reasoning_Chains and downstream sequences must conform to it (per STRAT-PRIMACY-001).

WHY #3 — Why does M4 consume Organizational_DNA and Market_Narrative, not just signals?

Because strategy is about character and context, not just events. The same signal demands a different strategy for a slow-consensus, governance-heavy account (SNB pattern) than for a fast-mover, and a different framing inside an accelerating narrative than a vacuum. M4 conditions its reasoning on Tier 6 judgment.

NEW RULE — M4-CONDITION-001: M4 conditions strategy on Organizational_DNA (pacing, consensus difficulty, displacement feasibility) and Market_Narrative alignment/phase (framing, urgency). A strategy contradicting known DNA (e.g. fast-close on a slow-consensus account) is flagged for justification (DNA-PRIMACY-001).

WHY #4 — Why does M4 run the seven-step reasoning chain with stored confidence per step?

Because failure can occur at any step (wrong pain inference, wrong persona) and partial reasoning must be rejected. Step-level storage with confidence lets the system locate exactly where reasoning weakened and route low-confidence chains to human review.

NEW RULE — M4-CHAIN-001: M4 executes the mandatory reasoning sequence (signal interpretation → account research → pain inference → persona analysis → strategy decision → [handoff to M7 generation] → QC) storing each step's input/output/confidence. Any skipped or low-confidence step invalidates the chain (REASON-STEPPS-001).

WHY #5 — Deepest reason M4 exists?

Because the difference between an intelligent system and a template engine is whether there is a defensible thought process behind every action. M4 is the strategic mind — it ensures the system always knows WHY it is about to act, can explain it, and can improve it. It is reasoning made institutional.

NEW RULE — M4-PRIMACY-001: No message, sequence step, or strategic action may exist without an M4-produced, complete, QC-passing Reasoning_Chain conforming to the active Account_Strategy. This is enforced at the data layer, not by convention.

① Workflow Logic (step-by-step)

1. Enrichment-ready event from M3 (or strategy-review trigger) received
2. Step 1 Signal interpretation: what happened, why it matters to Decimal
3. Step 2 Account research: priorities, initiative, incumbency, DNA, narrative alignment
4. Step 3 Pain inference: operational pains implied by signals + research
5. Step 4 Persona analysis: target committee member motivations, decision style, risk posture
6. Step 5 Strategy decision: angle, value prop, ask, connector path, timing intent
7. Build/refresh Account_Strategy; check readiness gates + DNA/narrative conformance
8. Persist Reasoning_Chain (steps + confidence) + Account_Strategy; hand off to M7 for generation

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
• Enrichment + committee/initiative/vendor intelligence • Three_Score_Record • Buying_Window • Organizational_DNA • Market_Narrative • Relationship_Edge / Connector (path) • Market_Interaction_History • Decision_Pattern (advisory) • Rule_Registry	• Reasoning_Chain • Account_Strategy • Risk_Assessment (request) • Capability_Gap (if cannot reason confidently) • System_Event

③ Decision Nodes & Gates

Decision node	Branch logic
All reasoning steps complete + above confidence floor?	NO → reject chain, declare gap or route to human; YES → continue
Readiness gates clear for solution-selling?	NO → strategy stays in PREPARATION (discovery only) per STRAT-READINESS-001
Strategy conforms to DNA + narrative?	NO → flag mismatch, require justification
Warm connector path exists for max-tier target?	YES → prefer it (CONNECTOR-PRIMACY-001); NO → justify cold approach

④ Module Boundary & Orchestration

M4 reasons and strategizes ONLY. It does NOT write message copy (M7), execute (M8), allocate portfolio capital (M6), or set timing schedules (M5) — it decides the angle and feeds them. It consumes scores/windows/DNA; it does not compute them. Orchestration: triggered by M3 events; reasoning runs in an AI service, not n8n.

M5 — TIMING & WINDOW

Owns the Buying_Window lifecycle and all 'now vs. later' decisions — operationalizing timing as the primary competitive weapon.

WHY #1 — Why is timing a dedicated module rather than a field on the account?

Because timing has its own lifecycle, state machine, and prediction logic. A window opens, peaks, decays, and closes; the optimal action point is near peak, not first detection. A dedicated module manages this state machine and computes the Window Multiplier that the opportunity equation consumes.

NEW RULE — M5-WINDOW-001: M5 owns Buying_Window creation, state transitions (OPENING/PEAK/CLOSING/CLOSED), and the Window Multiplier. Windows open on qualifying signals intersecting account state (and narrative phase); the multiplier peaks at PEAK and decays toward CLOSING (per WINDOW-STATE-001).

WHY #2 — Why predict window closure rather than just observe it?

Because reacting after closure is too late — that is the difference between timing as weapon and timing as luck. M5 must forecast decay so action is scheduled before the window shuts, raising execution urgency as closure approaches.

NEW RULE — M5-PREDICT-001: M5 maintains estimated_closure with confidence per window, updated as signals age and via M11 timing calibration. Approaching closure raises execution urgency in the decision sequence and can pre-empt lower-priority work (WINDOW-PREDICT-001).

WHY #3 — Why does M5 read Market_Narrative phase, not just account signals?

Because a window opened by a signal riding an accelerating narrative is stronger than the same signal in a vacuum. Narrative phase amplifies or dampens the window — timing is about the market's tempo, not just the account's.

NEW RULE — M5-NARRATIVE-001: M5 amplifies the Window Multiplier when the opening signal aligns with an ACCELERATING Market_Narrative and dampens it for a DECLINING one (per NARR-PHASE-001). Narrative phase is a first-class input to window strength.

WHY #4 — Why does M5 gate execution rather than just inform scoring?

Because the decision sequence has a distinct timing step (step 5) that asks 'now or later?' independently of 'how attractive?'. M5 answers it: even a high-opportunity account may wait if its window is merely opening, and a closing window may jump the queue.

NEW RULE — M5-GATE-001: M5 provides the timing decision in the Universal Decision Sequence. No 'now vs later' decision may be made without consulting the account's active window state (WINDOW-PRIMACY-001). Closing windows can re-prioritize execution order ahead of higher raw scores.

WHY #5 — Deepest reason M5 exists?

Because in enterprise selling, being right at the wrong time is being wrong. M5 is the system's sense of timing — the discipline that turns 'who to contact' into 'who to contact NOW.' It is timing-as-weapon made into running code.

NEW RULE — M5-PRIMACY-001: M5 is the authority on engagement timing. Execution (M8) may not start or advance a sequence step against M5's window/timing guidance without explicit override. Timing is enforced, not advisory.

① Workflow Logic (step-by-step)

1. Window-opening signal blessed (new license, exec hire, funding, mandate, renewal approaching)
2. Check narrative alignment + account state → create or update Buying_Window
3. Set open_state and compute Window Multiplier; link trigger signals
4. Forecast estimated_closure + confidence
5. Emit multiplier to M2 (opportunity equation) + timing guidance to M6/M8
6. On signal aging / new events → transition state, recompute multiplier + closure
7. As closure approaches → raise execution urgency event
8. On closure → set CLOSED, retain for M11 window-prediction training

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
• Signal (window-opening, blessed) • Market_Narrative (phase) • Organizational_DNA (decision velocity) • Vendor_Intelligence (renewal dates) • Initiative (maturity) • Rule_Registry (decay curves, multiplier params)	• Buying_Window • System_Event (window open/transition/close, urgency) • feeds Three_Score_Record via M2

③ Decision Nodes & Gates

Decision node	Branch logic
Qualifying signal cause present?	NO → no window (invalid per WINDOW-CAUSE-001); YES → open
Window state?	OPENING → patience; PEAK → act; CLOSING → urgency/jump queue; CLOSED → no independent trigger
Narrative accelerating?	YES → amplify multiplier; DECLINING → dampen
Closure imminent + account ready?	YES → emit high-urgency execution event

④ Module Boundary & Orchestration

M5 owns timing and windows ONLY. It does NOT score attractiveness (M2), decide strategy (M4), or allocate capital (M6) — it tells them WHEN. It computes the Window Multiplier but does not apply the full opportunity equation (that is M2). Orchestration: event-driven service subscribed to blessed signals; n8n routes window events.

M6 — PORTFOLIO ALLOCATION

Decides how to spend finite system capital (attention, connector, brand, daily account starts) across all accounts each cycle — the leap from per-account optimization to portfolio strategy.

WHY #1 — Why allocate at the portfolio level rather than let each account pull capital as scored?

Because capital is finite and shared. If every HOT account independently triggered full outreach, the system would overspend attention, fatigue connectors, and risk brand. A portfolio module rations scarce capacity across accounts by expected return under hard constraints.

NEW RULE — M6-ALLOCATE-001: M6 produces a Portfolio_Allocation each cycle, distributing finite capital (attention slots, connector asks, daily account starts) across accounts. No account consumes system capital outside the active allocation (PORT-PRIMACY-001); exhausted budgets queue work to the next cycle.

WHY #2 — Why rank by EROA rather than raw Decision Score?

Because a high score that costs enormous capital can be worse than a cheaper moderate one. EROA = Effective Opportunity / Total Capital Cost normalizes by cost, maximizing total portfolio return under constraint. Negative-EROA accounts are excluded regardless of attractiveness.

NEW RULE — M6-EROA-001: M6 ranks accounts by EROA and excludes negative-EROA accounts for the cycle (PORT-EROA-001). Capital cost aggregates attention, connector, brand, and system costs per SCARCITY-001.

WHY #3 — Why does M6 consume the Market_Dependency_Graph?

Because opportunities are correlated, not independent — winning SNB lifts SAB/Riyad/Alinma. A keystone account that unlocks references can beat a higher-standalone-EROA isolated account. M6 must value an account partly by its effect on the rest of the portfolio (true portfolio theory).

NEW RULE — M6-DEPENDENCY-001: M6 incorporates Market_Dependency_Graph effects: an account's effective value includes expected lift to dependent accounts (keystone value), and negative dependencies (reputation contagion) raise its risk weight. Dependency influence is capped while edges are ASSUMED (DEP-CONFIDENCE-001).

WHY #4 — Why enforce concentration limits and reconcile spend each cycle?

Because all-capital-on-one-whale risks total loss and over-concentration on one connector burns the asset. Limits manage portfolio risk; reconciliation of planned vs. actual spend catches overspend and feeds better future allocation.

NEW RULE — M6-DISCIPLINE-001: M6 enforces concentration limits per account/connector (PORT-DIVERSIFY-001) and reconciles planned vs actual capital spend each cycle (PORT-RECONCILE-001). Systematic overspend triggers M11 cost-model recalibration.

WHY #5 — Deepest reason M6 exists?

Because the system must behave like a disciplined investor of finite, irreplaceable capital — not a sprayer of attention. M6 embodies scarcity economics: the recognition that attention is the binding constraint and must be invested where total, correlated return is highest. It is portfolio judgment made operational.

NEW RULE — M6-PRIMACY-001: Sequence starts (M8) and connector asks draw down M6-allocated budgets. M6 is the sole authority on cross-account capital distribution; modules may request capital but not self-allocate beyond their cycle budget.

① Workflow Logic (step-by-step)

1. Cycle start (or significant re-rank event) → gather candidate accounts with Decision Scores
2. Compute Total Capital Cost per account (attention/connector/brand/system)
3. Compute EROA; exclude negative-EROA accounts
4. Apply Market_Dependency_Graph: add keystone lift, raise risk for negative blast-radius
5. Apply concentration limits per account/connector
6. Produce Portfolio_Allocation (capital per account, connector ask budgets, daily starts)
7. Release allocated budgets to M8 (execution) and connector asks to M10 paths
8. End of cycle: reconcile planned vs actual; emit overspend signals to M11

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
• Three_Score_Record (Decision Score) • Market_Dependency_Graph • Connector (budgets, reciprocity) • Trust_Capital_Ledger (autonomy bounds) • Buying_Window (urgency from M5) • Rule_Registry (caps, cycle params)	• Portfolio_Allocation • Connector budget draws • System_Event (allocation, reconciliation)

③ Decision Nodes & Gates

Decision node	Branch logic
EROA positive?	NO → exclude this cycle; YES → eligible
Within concentration cap?	NO → trim allocation or require human approval; YES → allocate
Connector reciprocity healthy?	NO → block connector ask until rebalanced (CONNECTOR-RECIPROCITY-001)
Closing window urgency from M5?	YES → may jump allocation priority

④ Module Boundary & Orchestration

M6 allocates capital across accounts ONLY. It does NOT score individual accounts (M2), decide per-account strategy (M4), or execute (M8). It consumes dependency/score/window inputs; it does not compute them. Orchestration: cycle-scheduled service (plus event re-rank); n8n triggers the cycle and routes budget releases.

M7 — AI GENERATION & QC

Generates the actual outreach copy from M4's reasoning chain and validates it against hallucination, tone, and enterprise-suitability checks before anything can be sent.

WHY #1 — Why is generation separate from reasoning (M4)?

Because reasoning decides WHAT to say and why; generation decides HOW to phrase it. Separating them means the same reasoning can produce email vs LinkedIn vs Arabic variants, and a phrasing failure never corrupts the underlying strategy. M7 is the voice; M4 is the mind.

NEW RULE — M7-SEPARATE-001: M7 generates message content strictly FROM an M4 Reasoning_Chain; it may not introduce strategy or facts absent from the chain. Each generated Message links its reasoning_chain_id, prompt_version, and model_id (MESSAGE-AI-PROVENANCE-001).

WHY #2 — Why must every generation pass QC before it can be queued?

Because hallucination is a named failure mode and a single fabricated fact can destroy credibility with an auditor-minded buyer. QC (hallucination score, factual grounding, tone, enterprise suitability) is a hard gate, not a suggestion.

NEW RULE — M7-QC-001: M7 runs mandatory QC on every draft: hallucination_score, factual-grounding against blessed enrichment, tone, and enterprise suitability. Drafts failing thresholds are rejected (status QC_REJECTED, retained for analysis) and never queued (REASON-QC-001).

WHY #3 — Why ground every claim against M0-blessed enrichment specifically?

Because a message may only assert what the system actually, verifiably knows. Grounding generation against blessed facts (not the model's parametric guesses) is how the system stays truthful with high-stakes buyers.

NEW RULE — M7-GROUND-001: Every factual claim in a generated message must trace to an M0-blessed Enrichment fact or Signal. Ungrounded claims are stripped or the draft is rejected. The model may not introduce unverifiable specifics (numbers, names, dates) absent from the intelligence base.

WHY #4 — Why version prompts and route generation through the registry?

Because phrasing performance must be measurable and reversible. Tying each message to a Prompt_Version enables A/B testing, outcome attribution, and instant rollback of a prompt that damages brand at scale.

NEW RULE — M7-PROMPT-001: M7 generates only via registered Prompt_Versions and approved AI_Model_Registry models for the decision type. Outcomes attribute to prompt_version for M11 optimization; prompts affecting max-tier contacts require human approval before activation (PROMPT-GOVERNANCE-001).

WHY #5 — Deepest reason M7 exists?

Because language is the system's product surface — the actual point of contact with the market — and it must be controlled, truthful, and improvable. M7 is where strategy becomes words, and where the system's voice is kept honest and on-brand.

NEW RULE — M7-PRIMACY-001: No Message exists without an M7 generation linked to a Reasoning_Chain and a registered Prompt_Version, having passed QC. Untracked or un-QC'd message creation is prohibited (MESSAGE-TRACE-001, PROMPT-PRIMACY-001).

① Workflow Logic (step-by-step)

1. Generation request from M4 with completed Reasoning_Chain + target channel/persona/language
2. Select registered Prompt_Version + approved model for the decision type
3. Generate draft content grounded strictly in chain + blessed enrichment
4. QC: hallucination score, factual grounding, tone, enterprise suitability, language correctness
5. Decision: PASS → status QUEUED (hand to M8) / FAIL → status QC_REJECTED (retain, optionally regenerate)
6. Persist Message (content, provenance, QC results, capital_ledger link)
7. Debit brand/relationship/connector capital ledgers on queue

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
• Reasoning_Chain (M4) • Enrichment (blessed, for grounding) • Contact.persona_profile • Prompt_Version • AI_Model_Registry • Account_Strategy (conformance) • Rule_Registry (QC thresholds)	• Message (DRAFT/QC_REJECTED/QUEUED) • Trust_Capital_Ledger draw • System_Event (generated, QC result)

③ Decision Nodes & Gates

Decision node	Branch logic
Reasoning chain complete + QC-passing?	NO → reject generation request (M4-PRIMACY-001)
Hallucination score within limit?	NO → reject draft
All claims grounded in blessed facts?	NO → strip claim or reject
Max-tier contact + prompt approved?	NO → hold for human prompt approval

④ Module Boundary & Orchestration

M7 generates and QC's copy ONLY. It does NOT decide strategy (M4), schedule/send (M8), or judge engagement (M9). It cannot introduce facts beyond the blessed intelligence base. Orchestration: AI service called after M4; n8n routes the queued message to M8.

M8 — OUTREACH EXECUTION

Executes paced, condition-aware, multi-channel outreach sequences within all hard limits — turning approved messages into actual market contact at the right cadence.

WHY #1 — Why is execution sequence-based rather than send-on-approval?

Because enterprise engagement is multi-touch over time with hard pacing constraints. A sequence enforces cadence, channel mix, gaps, and limits that no single send can. Bursting approved messages is spam; paced sequencing is selling.

NEW RULE — M8-SEQUENCE-001: M8 executes all outbound through Outreach_Sequence with hard limits: max email/LinkedIn touches per contact, minimum inter-touch gaps, the 14/10-day spreads, and max new accounts/day. Ad-hoc out-of-sequence sends are prohibited except logged human one-offs (SEQ-PRIMACY-001).

WHY #2 — Why re-evaluate each step at fire-time rather than execute the plan blindly?

Because state changes between planning and sending — a window closes, a contact leaves, sentiment drops, the strategy is revised. A step must re-check current state and chain freshness before firing, regenerating or cancelling if stale.

NEW RULE — M8-REEVAL-001: Before each step fires, M8 re-evaluates account/contact/window state, Account_Strategy conformance, and Reasoning_Chain freshness. Stale or invalidated steps are regenerated (via M4/M7) or cancelled, never sent blindly (SEQ-REEVAL-001).

WHY #3 — Why bind execution to M5 timing and M6 capital, not just per-contact limits?

Because per-contact limits aren't enough — the system has global capacity (daily starts) and timing imperatives (closing windows). Execution must respect both the portfolio budget from M6 and the timing guidance from M5.

NEW RULE — M8-CONSTRAINTS-001: M8 sequence starts draw down M6-allocated budgets and respect M5 timing guidance. When global capacity is exhausted, starts queue by Decision Score (not FIFO). Closing-window urgency from M5 can reorder the queue.

WHY #4 — Why does M8 enforce exit conditions as first-class logic?

Because a sequence that ignores responses is spam and burns brand. Execution must halt immediately on reply, negative sentiment, window closure, or do-not-contact — exit logic is core to the module, not an afterthought.

NEW RULE — M8-EXIT-001: M8 enforces sequence exit conditions (reply, negative sentiment, window closed, do-not-contact, fatigue). Any exit immediately pauses/terminates remaining steps. Contact-level fatigue and Market_Interaction_History gates are checked before every send (CONTACT-GOVERNANCE-001).

WHY #5 — Deepest reason M8 exists?

Because execution is where strategy meets reality and brand capital is actually spent. M8 operationalizes timing and restraint — paced, condition-aware contact rather than a burst of messages. It is the discipline that the system always sends the right touch, to the right person, at the right cadence, or not at all.

NEW RULE — M8-PRIMACY-001: All market contact flows through M8 under these constraints. No module may dispatch outbound messages directly; M8 is the sole execution authority and the sole writer of SENT status.

① Workflow Logic (step-by-step)

1. Queued message + Outreach_Sequence plan received; sequence start checked against M6 budget + M5 timing
2. Pre-send gate: re-evaluate state, strategy conformance, chain freshness, contact fatigue/DNC/history
3. If stale → route back to M4/M7 for regeneration; if exit condition met → terminate
4. Dispatch via channel tool (email: Instantly/Smartlead; LinkedIn: Heyreach) respecting gaps
5. Record SENT Message; debit capital; emit sent System_Event
6. Schedule next step with required gap; await engagement (M9) or exit
7. On engagement/exit signal from M9 → pause/advance/terminate sequence accordingly

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
------------------	-------------------

• Message (QUEUED) • Outreach_Sequence • Portfolio_Allocation (budget) • Buying_Window (timing from M5) • Contact.contact_flags • Market_Interaction_History • Account_Strategy • Rule_Registry (limits, gaps)	• Message (SENT) • Outreach_Sequence (status/step advance) • Trust_Capital_Ledger / Connector draws • System_Event (sent, paused, exited)
---	--

③ Decision Nodes & Gates

Decision node	Branch logic
Sequence start within M6 budget?	NO → queue by Decision Score; YES → start
Pre-send state fresh + conformant?	NO → regenerate or cancel step
Contact fatigued / DNC / prior damage?	YES → block send (hard gate); NO → send
Exit condition met (reply/neg/window-closed)?	YES → terminate remaining steps

④ Module Boundary & Orchestration

M8 paces and dispatches ONLY. It does NOT generate copy (M7), decide strategy (M4), or interpret responses (M9). It is the sole sender. Orchestration: n8n coordinates channel-tool integrations and step scheduling; pacing/exit logic lives in the execution service, not n8n.

M9 — ENGAGEMENT TRACKING

Captures recipient-side actions (opens, clicks, replies, profile views, meeting accepts) as inbound signals, feeding scoring, windows, and the learning loop in near-real-time.

WHY #1 — Why is engagement a distinct module from execution (M8)?

Because sending and receiving are different actors and timelines. M8 records what we send; M9 records what they do. Conflating them loses the asymmetry between our action and their reaction, which is the entire signal of whether outreach worked.

NEW RULE — M9-CAPTURE-001: M9 captures recipient/third-party-observed actions as Engagement_Event entities (action_type, observed_via, occurred_at), distinct from M8's outbound Messages. Where traceable, each links triggering_message_id (ENGAGE-ATTRIBUTION-001).

WHY #2 — Why treat engagement as a first-class signal that retriggers scoring?

Because engagement IS inbound buying intent — a reply or repeated profile views should immediately update the account's score and may open or widen a window. Treating engagement as passive analytics would waste the strongest real-time intent signal the system has.

NEW RULE — M9-SIGNAL-001: High-intent Engagement_Events (reply, meeting-accept, repeated content interaction) retrigger M2 scoring and notify M5 to open/widen a Buying_Window. Engagement is a first-class signal class, processed near-real-time, not batched (ENGAGE-TRIGGER-001).

WHY #3 — Why store engagement as discrete events rather than aggregate counts?

Because timing and sequence carry meaning — three opens in an hour signal different intent than three over three weeks. Discrete, timestamped events preserve the temporal pattern (velocity, clustering, recency) that aggregates destroy.

NEW RULE — M9-GRANULAR-001: M9 stores discrete timestamped Engagement_Events, never pre-aggregated counters. Velocity/clustering/recency are computed at read time (ENGAGE-GRANULARITY-001). Sentiment on replies updates Contact.interaction_lineage and Market_Interaction_History.

WHY #4 — Why does M9 feed both M8 (sequence control) and M11 (learning)?

Because engagement controls the present and teaches the future. It tells M8 to pause/advance/exit a sequence now, and it feeds M11's attribution so the system learns which content and timing converted. The same event serves real-time control and long-run learning.

NEW RULE — M9-FEED-001: M9 emits engagement to M8 (sequence exit/advance), M2 (rescore), M5 (window), and M11 (Feedback_Loop_Entry). Absence of engagement after N touches is itself emitted as a learning signal (triggers sequence pause / strategy review per ENGAGE-LEARNING-001).

WHY #5 — Deepest reason M9 exists?

Because it is the system's only feedback from reality about whether its strategy worked — the sensory return path that makes the system a learning loop, not a one-way broadcaster. M9 is what makes this intelligence rather than automation.

NEW RULE — M9-PRIMACY-001: Every recipient action that the system can observe is captured by M9. No other module infers engagement independently; M9 is the sole authority on engagement state, feeding all reactive and learning modules.

① Workflow Logic (step-by-step)

1. Engagement webhook/poll received from channel tools (open, click, reply, profile view, meeting accept)
2. Normalize → Engagement_Event; classify intent (HIGH/MEDIUM/LOW)
3. Link triggering_message_id where traceable; persist discrete event
4. If reply: run sentiment; update Contact.interaction_lineage + Market_Interaction_History
5. Emit to M8 (exit/advance), M2 (rescore), M5 (window), M11 (feedback)
6. If positive intent → trigger M10 human handoff path
7. If no engagement after N touches → emit learning signal (pause/review)

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
• Channel tool webhooks (email/LinkedIn) • Message (for attribution link) • Rule_Registry (intent classification, N-touch thresholds)	• Engagement_Event • Contact.interaction_lineage • Market_Interaction_History • System_Event • Feedback_Loop_Entry (via M11)

③ Decision Nodes & Gates

Decision node	Branch logic
Engagement traceable to a Message?	YES → link for attribution; NO → store unattributed
Intent class HIGH?	YES → rescore + window + possible handoff; LOW → record only
Reply sentiment negative?	YES → sequence exit + update history (do-not-return)
Positive buying intent?	YES → trigger M10 handoff

④ Module Boundary & Orchestration

M9 observes and classifies recipient actions ONLY. It does NOT send (M8), score (M2), or run handoff (M10) — it triggers them. It is the sole authority on engagement state. Orchestration: n8n receives channel webhooks and routes to M9's service; classification logic lives in the service.

M10 — HUMAN HANDOFF & REVIEW

Transitions qualified engagement to human sales teams and governs all human review of AI decisions — the hinge of AI-human collaboration where authority is shared and trust is calibrated.

WHY #1 — Why is human handoff/review a dedicated module rather than a notification?

Because AI-human collaboration (Principle 3) requires structured capture of human judgment, not a fire-and-forget alert. M10 owns both directions: handing qualified deals TO humans, and bringing human review decisions BACK into the system to calibrate AI autonomy.

NEW RULE — M10-RECORD-001: M10 records every human review as an immutable Human_Review_Record (artifact, decision approve/edit/reject, edits, rationale, reviewer, latency). It routes qualified engagement to the right human role (SDR/AE/technical) by trigger type (REVIEW-RECORD-001).

WHY #2 — Why require rationale on every review decision, not just the verdict?

Because a bare approve/reject teaches the system nothing and hides rubber-stamping. Rationale is the training signal that improves future AI proposals and lets the system detect oversight that has gone hollow.

NEW RULE — M10-RATIONALE-001: M10 requires a rationale for approvals and rejections. Rationale-free verdicts are flagged; high rates of instant rationale-free approvals are detected as rubber-stamping and reduce the effective trust granted to that reviewer (REVIEW-RUBBERSTAMP-001).

WHY #3 — Why does M10 feed the Trust_Capital_Ledger?

Because autonomy must be earned and calibrated per decision type. Consistent human agreement should expand AI autonomy; disagreement should contract it, even reinstating mandatory review. M10 is how reviews translate into changing AI authority.

NEW RULE — M10-TRUST-001: M10 review outcomes update the Trust_Capital_Ledger per decision type: sustained agreement raises earned autonomy, disagreement lowers it and can trip the autonomy circuit-breaker back to mandatory review (REVIEW-TRUST-001, TRUST-DEBIT-001).

WHY #4 — Why gate high-risk/max-tier actions through M10 before execution?

Because some actions (max-tier contacts, high blast-radius, low-confidence reasoning, compliance-sensitive) carry consequences that exceed the AI's earned autonomy. M10 ensures a human reviews these before they reach the market.

NEW RULE — M10-GATE-001: Actions above defined risk/autonomy thresholds (max-tier contact, high blast-radius, compliance-sensitive, low-confidence chain) require an M10 Human_Review_Record before M8 may execute. The system never self-grants authority beyond its ledger-recorded autonomy (REVIEW-PRIMACY-001).

WHY #5 — Deepest reason M10 exists?

Because the human must remain meaningfully in command, and trust between human and AI must be built on evidence, not assumption. M10 is the mechanism of shared authority and preserved accountability — the reason the system earns independence honestly rather than seizing it.

NEW RULE — M10-PRIMACY-001: M10 is the sole authority on human-AI authority transfer and review capture. AI autonomy levels are defined only by the Trust_Capital_Ledger as updated through M10; no module may act beyond its earned, M10-reviewed autonomy.

① Workflow Logic (step-by-step)

1. Positive-intent trigger from M9 (reply, meeting request, pricing/security question) → route to role
2. OR high-risk/max-tier action from M4/M7/M8 → enqueue for mandatory review
3. Present artifact (reasoning chain / message / strategy) + context to human
4. Capture decision: approve / edit / reject + mandatory rationale
5. On approve → release action to M8; on edit → revised artifact to M8; on reject → return to M4/M7

6. Write Human_Review_Record; update Trust_Capital_Ledger for the decision type
7. Monitor review patterns (latency, approval rate) for rubber-stamping; alert governance
8. Create CRM task / assign owner for handed-off deals

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
- Engagement_Event (handoff triggers) - Reasoning_Chain / Message / Account_Strategy (review) - Risk_Assessment - Trust_Capital_Ledger (current autonomy) - Rule_Registry (review thresholds)	- Human_Review_Record - Trust_Capital_Ledger (credit/debit) - CRM task / owner assignment - System_Event (handoff, review, trust change)

③ Decision Nodes & Gates

Decision node	Branch logic
Action above autonomy/risk threshold?	YES → mandatory review before execute; NO → autonomous
Rationale provided?	NO → block submission, flag; YES → record
Reviewer rubber-stamping pattern?	YES → discount trust impact + alert governance
Positive buying intent?	YES → route to SDR/AE/technical by trigger type

④ Module Boundary & Orchestration

M10 governs human handoff and review ONLY. It does NOT generate, score, or execute — it gates and captures. It is the sole authority on AI autonomy changes. Orchestration: n8n routes triggers and notifications to humans (Slack/CRM); review capture and trust-ledger logic live in the service.

M11 — LEARNING & ANALYTICS

Closes the loop from action to outcome to improvement — attributing results, recalibrating weights/timing/prompts, and maintaining performance baselines, all evidence-based and reversible.

WHY #1 — Why is learning a dedicated module rather than scattered tweaks?

Because improvement must be auditable, bounded, and reversible. A dedicated module ensures every adjustment cites its evidence, respects stability limits, never touches immutable principles, and can be rolled back. Scattered self-modification is how a system silently drifts.

NEW RULE — M11-CENTRAL-001: M11 is the sole authority for system self-adjustment. Every change (scoring weight, threshold, timing curve, prompt promotion, pattern update) is a Feedback_Loop_Entry citing evidence, rate-limited and reversible (FEEDBACK-RECORD-001, FEEDBACK-PRIMACY-001).

WHY #2 — Why attribute outcomes multi-touch rather than last-touch?

Because enterprise wins come from many touches over time; last-touch attribution misleads and would teach the system the wrong lessons. Multi-touch credit reveals which sequence of decisions actually built the outcome.

NEW RULE — M11-ATTRIBUTE-001: M11 produces Outcome_Attribution with multi-touch credit across contributing Reasoning_Chains, Prompt_Versions, timing, and connector paths (ATTR-MULTITOUCH-001). Failures are attributed with equal rigor (ATTR-FAILURE-001).

WHY #3 — Why does M11 consume Counterfactual_Records at discounted weight?

Because the system should learn from roads not taken (would waiting have won?) but must never treat an estimate as fact. Counterfactual regret informs patience gates and timing curves at a confidence-discounted weight, never auto-driving decisions.

NEW RULE — M11-COUNTERFACTUAL-001: M11 ingests Counterfactual_Records at a confidence-discounted weight relative to observed outcomes (CF-FEEDBACK-001). They may adjust patience gates and timing curves within bounds but never override observed-fact learning or immutable meta-principles.

WHY #4 — Why maintain explicit performance baselines and detect regression?

Because improvement is meaningless without a reference, and silent regressions (including from learning gone wrong) erode performance. Baselines per metric and segment let M11 prove a change helped and auto-roll-back ones that hurt.

NEW RULE — M11-BASELINE-001: M11 maintains Performance_Baselines per metric and segment, continuously compares live metrics, and on significant regression alerts and can auto-pause/roll-back the responsible change (BASELINE-REGRESSION-001). Re-baselining is an explicit logged action (BASELINE-REBASE-001).

WHY #5 — Deepest reason M11 exists?

Because the system's deepest purpose is to accumulate judgment, not just activity. M11 is the disciplined heartbeat of learning — evidence-based, bounded, auditable, reversible improvement that makes the system measurably wiser over time without betraying its principles.

NEW RULE — M11-PRIMACY-001: All self-improvement flows through M11. Silent, unlogged, or unbounded self-modification anywhere in the system is prohibited. M11 may never alter immutable meta-principles; it operates only within governed operational bounds (FEEDBACK-STABILITY-001).

① Workflow Logic (step-by-step)

1. Outcome event (reply/meeting/pilot/win/loss/opt-out) or counterfactual resolution received
2. Build Outcome_Attribution: multi-touch credit to chains/prompts/timing/connectors
3. Ingest Counterfactual_Records (discounted weight) for engage/wait + timing lessons
4. Compare results against Performance_Baseline (per metric + segment)
5. Derive adjustment proposals (weights, thresholds, timing curves, prompt/pattern updates)
6. Validate against significance + stability guards; reject noise-driven changes
7. Write Feedback_Loop_Entry (evidence-linked, reversible); apply within bounds via Rule_Registry
8. Monitor for regression; auto-rollback responsible change if baseline breached

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
• Outcome_Attribution inputs (Engagement_Event, Message, Reasoning_Chain) • Counterfactual_Record • Timing_Record • Human_Review_Record (patterns) • Performance_Baseline • Decision_Pattern • Rule_Registry	• Outcome_Attribution • Feedback_Loop_Entry • Decision_Pattern (update/decay) • Performance_Baseline (deliberate rebaseline) • Rule_Registry (governed weight/threshold changes) • Timing_Record calibration • System_Event

③ Decision Nodes & Gates

Decision node	Branch logic
Adjustment backed by significant evidence?	NO → reject (no noise-chasing per FEEDBACK-EVIDENCE-001)
Change within stability/rate limits?	NO → damp or defer; YES → apply

Decision node	Branch logic
Touches immutable meta-principle?	YES → forbidden, block
Live metric regressed vs baseline?	YES → alert + auto-rollback responsible change

④ Module Boundary & Orchestration

M11 learns and recalibrates ONLY within governed operational bounds. It does NOT make per-account live decisions (M2/M4/M6) or alter immutable principles. It proposes and applies bounded, reversible improvement. Orchestration: outcome events routed via n8n; attribution + recalibration logic in services; Rule_Registry writes are governed.

M12 — STRATEGIC MEMORY

Owns the Tier 6 entities — building and maintaining Organizational_DNA, Market_Narrative, Counterfactual_Record, and Market_Dependency_Graph: the system's accumulating judgment about why reality behaves as it does.

WHY #1 — Why is Strategic Memory a dedicated module, separate from analytics (M11)?

Because M11 learns WHAT happened (outcomes, weights); M12 learns WHY reality behaves as it does (character, narrative, regret, dependency). They have different update tempos and different jobs — M11 recalibrates fast operational parameters; M12 maintains slow-moving strategic priors that condition the whole system.

NEW RULE — M12-SEPARATE-001: M12 owns the Tier 6 entities and their slow, deliberate update cadence, distinct from M11's operational recalibration. M12 outputs condition upstream modules (M2/M4/M5/M6); it does not itself score, reason, time, or allocate (META-T6 admission test).

WHY #2 — Why does M12 maintain DNA as a slow prior rather than recompute it often?

Because character persists while state fluctuates. Recomputing DNA on every signal would let a turbulent quarter erase years of established character. M12 updates DNA only when deviations from the prior persist across multiple assessments.

NEW RULE — M12-DNA-001: M12 maintains Organizational_DNA as a slow prior, updating only when Organizational_Assessment deviations persist across multiple readings (DNA-PRIOR-001). It clusters DNA into reusable buying archetypes and assigns provisional archetypes to sparse accounts (DNA-ARCHETYPE-001).

WHY #3 — Why does M12 own market-level narrative tracking?

Because narrative is the only supra-account entity — it shapes the whole segment, not one account. A dedicated owner tracks narrative lifecycle and computes per-account alignment, feeding framing and timing across all accounts simultaneously.

NEW RULE — M12-NARRATIVE-001: M12 tracks Market_Narrative lifecycle (EMERGING/ACCELERATING/PEAK/DECLINING) and computes per-account Narrative_Alignment_Score, feeding M4 (framing) and M5 (window amplification). Narratives are shared, market-level, never account-owned (NARR-SCOPE-001).

WHY #4 — Why does M12 build counterfactuals and the dependency graph rather than M11?

Because both are judgment about alternatives and second-order effects — strategic, not operational. M12 generates Counterfactual_Records (regret, bounded by confidence) and maintains the Market_Dependency_Graph (correlated opportunities), handing both to M11 and M6 as discounted/capped inputs.

NEW RULE — M12-JUDGMENT-001: M12 generates Counterfactual_Records (labeled ESTIMATE, never auto-driving decisions per CF-CONFIDENCE-001) and maintains the Market_Dependency_Graph (edges ASSUMED until empirically earned per DEP-CONFIDENCE-001). Both degrade gracefully and are capped while low-confidence.

WHY #5 — Deepest reason M12 exists?

Because it is the module that makes the system wiser rather than merely busier — the institutional accumulation of judgment that persists across accounts, signals, and leadership. M12 is what turns the system from an action-optimizer into a developer of strategic judgment: a Cognitive Enterprise Operating System.

NEW RULE — M12-PRIMACY-001: M12 outputs (DNA, narrative alignment, regret, dependency) condition the decision stack but never silently override observed fact or human judgment (META-T6-003). Where Tier 6 estimate conflicts with raw activity, M12 surfaces the tension rather than trusting either blindly.

① Workflow Logic (step-by-step)

1. DNA: aggregate persistent Organizational_Assessment deviations → update DNA prior + archetype (slow cadence)
 2. Narrative: track external narrative signals → update lifecycle phase + per-account alignment
 3. Counterfactual: on material engage/wait + timing outcomes → generate regret estimate (with confidence)
 4. Dependency: on account outcomes (win/loss/reference) → update dependency edges + confidence/source
 5. Emit conditioning outputs: DNA → M4/M2, narrative → M4/M5, regret → M11, dependency → M6
 6. Surface tension events where Tier 6 estimate conflicts with raw recent activity

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
• Organizational_Assessment (deviations) • external narrative signals (via M1/M0) • Counterfactual inputs (decisions + outcomes) • Outcome_Attribution (for dependency) • Market_Interaction_History • Rule_Registry	• Organizational_DNA • Market_Narrative • Counterfactual_Record • Market_Dependency_Graph • System_Event (tension surfaced, prior updated)

③ Decision Nodes & Gates

Decision node	Branch logic
DNA deviation persistent across multiple assessments?	NO → keep prior (no overwrite); YES → update DNA
Narrative phase transition confirmed?	YES → update phase + realign accounts
Counterfactual confidence sufficient?	LOW → advisory only (never auto-drive)
Dependency edge empirically earned?	NO → keep ASSUMED + capped influence; YES → lift cap

④ Module Boundary & Orchestration

M12 owns strategic judgment (Tier 6) ONLY. It conditions M2/M4/M5/M6/M11 but never scores, reasons, times, allocates, or sends. It supplies priors and estimates, always bounded. Orchestration: slow-cadence services + event subscriptions; outputs published for upstream consumption.

M-Orch — ORCHESTRATION LAYER (n8n)

The event router and integration coordinator that connects modules, schedules triggers, and integrates external tools — explicitly holding NO business logic.

WHY #1 — Why have a separate orchestration layer at all?

Because modules must be coordinated, scheduled, and connected to external tools without each module knowing about all others. n8n is the nervous system's wiring — routing events, scheduling polls, and integrating channel/CRM tools — so modules stay decoupled and independently changeable.

NEW RULE — ORCH-ROUTER-001: n8n routes `System_Events` between modules, schedules time-based triggers (poll cadences, allocation cycles), and integrates external tools (email, LinkedIn, CRM, enrichment APIs). Modules communicate via events through the orchestrator, not direct coupling.

WHY #2 — Why must n8n hold NO business logic?

Because scoring, reasoning, timing, and allocation logic must be versioned, testable, and governed in services and the `Rule_Registry` — not buried in opaque workflow nodes. Mixing logic into orchestration makes it untestable and ungovernable.

NEW RULE — ORCH-NOLOGIC-001: n8n contains orchestration ONLY: routing, scheduling, integration. It must NOT contain scoring, reasoning, AI generation, conflict resolution, or any business logic — those live in services governed by the `Rule_Registry` (per the original n8n philosophy).

WHY #3 — Why route everything through `System_Events` rather than direct calls?

Because the event log is the system's audit spine and coordination mechanism at once. Routing via events means every cross-module action is traceable, causally linked, and reconstructable — and the denormalized caches can always be rebuilt from the log.

NEW RULE — ORCH-EVENT-001: All cross-module coordination flows through `System_Events` (immutable, causally linked per SYSEVENT-COORD-001). The orchestrator subscribes/publishes events; any cache must be reconstructable from the event log (SYSEVENT-PRIMACY-001).

WHY #4 — Why does the orchestrator enforce the pipeline order (M0 first, etc.)?

Because governance and reasoning gates only work if they cannot be bypassed. The orchestrator wires the mandatory sequence — M1 detect → M0 bless → M2 score → M3 enrich → M4 reason → M7 generate → M10 review (if gated) → M8 send → M9 engage → M11/M12 learn — so no stage can be skipped.

NEW RULE — ORCH-PIPELINE-001: The orchestrator enforces mandatory pipeline gates: no scoring before M0 blessing, no generation before M4 reasoning, no send before M8 (and M10 review where gated). These ordering constraints are wired in orchestration and cannot be bypassed by any module.

WHY #5 — Deepest reason the orchestration layer exists?

Because a system of independent intelligent modules needs a reliable, traceable nervous system to make them act as one coherent whole — without coupling them or hiding logic where it can't be governed. n8n is the connective tissue that keeps the system both modular and coordinated.

NEW RULE — ORCH-PRIMACY-001: The orchestrator is coordination infrastructure, not an actor. It may route, schedule, and integrate, but it never makes business decisions. Authority always rests with the modules and the `Rule_Registry`, never the orchestration layer.

① Workflow Logic (step-by-step)

1. Subscribe to `System_Events` from all modules
2. Schedule time-based triggers (signal poll cadences, M6 allocation cycle, baseline checks)
3. On event → route to the correct subscribing module(s) per pipeline rules
4. Integrate external tools: enrichment APIs, email (Instantly/Smartlead), LinkedIn (Heyreach), CRM (HubSpot)
5. Enforce mandatory ordering gates (M0/M4/M8/M10) — reject out-of-order requests
6. Log every routing action as part of the `System_Event` spine

② Entities Consumed / Written

Reads (consumes)	Writes (produces)
• System_Event (all) • Rule_Registry (schedules, routing/pipeline rules)	• routed events / external tool calls • System_Event (routing records)

③ Decision Nodes & Gates

Decision node	Branch logic
Pipeline order satisfied (e.g. M0 blessed before M2)?	NO → reject/hold; YES → route
Contains business logic?	Forbidden — logic belongs in services
External tool integration auth ok?	NO → backoff/alert; YES → call

④ Module Boundary & Orchestration

M-Orch coordinates ONLY (route/schedule/integrate). It holds NO business logic and makes NO business decisions. It enforces ordering but does not interpret content. Authority rests with modules + Rule_Registry.

SECTION 3 — COMPLETION SUMMARY

All 14 modules excavated via full 5-Whys, each documented with workflow logic, entity I/O, decision gates, and boundaries. The module set operationalizes every principle and entity from Sections 1–2: event-driven detection (M1), governance-first data quality (M0), three-score prioritization (M2), account-centric reasoning (M4), timing-as-weapon (M5), portfolio theory (M6), truthful generation (M7), paced execution (M8), engagement-as-signal (M9), human-AI trust calibration (M10), bounded learning (M11), and accumulating judgment (M12) — wired by a logic-free orchestration layer.

Modules excavated	14
5-Whys analyzed	70
New module rules	70
New modules from entity model	M5 Timing · M6 Portfolio · M12 Strategic Memory
Mandatory pipeline gates	M0 bless · M4 reason · M8 send · M10 review

Running total across the specification: Section 1 (~409 rules → 25 meta-principles + ~230 operational + 8-step sequence) · Section 2 (37 entities, 6 tiers, ~189 entity rules) · Section 3 (14 modules, 70 module rules). Next: Section 4 — Integrations & External Interfaces, or Section 5 — Edge Cases & Failure Modes.